

Capstone 3 — Domain B: Order Exception Resolution Agent

Build a ReAct agent that investigates B2B order exceptions — shipment delays, pricing discrepancies, partial deliveries, quality holds — by reasoning across ERP, warehouse, carrier, and contract data to propose resolutions with customer communication drafts.

PREREQUISITES

Complete M03 (Prompt Engineering), M04 (Structured Output), M05 (Function Calling), M06 (Multi-Tool Orchestration), M12 (ReAct Agent Loop), and M13 (Planning & Task Decomposition) before starting this capstone. You should be comfortable defining tool schemas, handling `tool_use` / `tool_result` message flows, orchestrating multiple tools, and building agentic loops that check `stop_reason`.

Difficulty: ★★★☆☆ — **Time estimate:** 90 minutes – 2 hours — **Steps:** 8–12 numbered steps with code/run/output/checkpoint

Project Brief

BUSINESS CONTEXT

A B2B operations analyst receives 20–30 order exceptions per day. Each exception is a puzzle: the customer reports “my order didn’t arrive,” but the root cause could be a carrier weather delay in Memphis, an inventory shortfall at the warehouse, a quality hold on the SKU, or a pricing discrepancy that held the shipment. The analyst must check 3–4 different systems (ERP, WMS, carrier tracking, contract database) to diagnose the problem.

The pain isn’t just the investigation time (15–30 minutes per exception) — it’s the *branching logic*. A shipment delay requires checking the carrier API. But if the carrier says “delivered,” the analyst pivots to warehouse confirmation. If the warehouse says “quality hold,” the analyst pivots again to the quality system. Each intermediate finding redirects the investigation. A linear tool-calling agent can’t handle this — it needs to *reason* about what to check next.

Your agent uses the ReAct pattern to investigate exceptions dynamically: it reads the exception description, reasons about which system to check first, adapts based on what it finds, and ultimately proposes a resolution with a drafted customer notification.

WHAT YOU WILL BUILD

A ReAct agent with 6 tools that:

- Accepts an order exception (type, PO number, customer description)
- Pulls order details from the ERP (`get_order_details`)
- Checks warehouse inventory levels (`query_warehouse_inventory`)
- Tracks shipments via carrier API (`track_shipment`)
- Verifies contract pricing (`get_contract_pricing`)
- Checks for quality holds (`check_quality_hold_status`)
- Drafts a customer notification (`draft_customer_notification`)

Skills practiced: ReAct loop (M12), multi-tool orchestration (M06), planning (M13), branching logic, error recovery, structured output.

Optional stretch goal: Add a `calculate_cost_impact` tool that estimates the financial impact of the resolution (e.g., expedited shipping cost, discount credit). See the section for the scaffolded version.

Domain Glossary

Order Exception

Any deviation from expected order flow: delayed shipment, pricing mismatch, partial delivery, quality hold. Requires multi-system investigation to diagnose and resolve.

Root Cause Analysis

Systematic investigation to determine WHY an exception occurred. The agent checks ERP, WMS, carrier, and contract data to trace the failure point.

WMS

Warehouse Management System — tracks inventory levels, pick/pack status, and warehouse operations. Source of truth for what's physically available to ship.

Quality Hold

A temporary block on shipping a product batch due to a quality concern (defect reports, supplier issue, inspection failure). Items on hold cannot be shipped until released.

Carrier Exception

A delay or issue reported by the shipping carrier: weather delay, customs hold, address correction, damage in transit. The carrier tracking API reports these as exception events.

Pricing Discrepancy

A mismatch between the price on the order and the customer's contract price. Can result from stale pricing, wrong tier applied, or contract expiration.

SLA Violation

When a delivery or service commitment is missed. Example: contract guarantees 5-day delivery but shipment took 8 days. May trigger credits or penalties.

RMA

Return Merchandise Authorization — a formal process for returning defective or incorrect items. Requires an RMA number before the customer can ship items back.

Architecture

REACT TRACE — INVESTIGATING A SHIPMENT DELAY

EXCEPTION TYPES — FOUR INVESTIGATION PATHS

TOOL ARSENAL — 6 INVESTIGATION TOOLS

Mock Data Specification

EXCEPTION + DATA SOURCES

EXCEPTION + DATA SOURCES

```
// — Exception Input —————
{
  "exception_id": "EXC-2024-0456",
  "po_number": "PO-2024-11234",
  "exception_type": "shipment_delay",
  "reported_by": "customer",
  "reported_date": "2024-03-10",
  "description": "Customer reports order not received by expected delivery date of 2024-03-08."
}

// — Order Details (from ERP) —————
{
  "po_number": "PO-2024-11234",
  "customer": "Acme Manufacturing",
  "line_items": [
    { "line": 1, "sku": "SKU-4892", "qty_ordered": 50, "qty_shipped": 50,
      "qty_delivered": 0, "unit_price": 495.00 }
  ],
  "total_value": 24750.00
}

// — Carrier Tracking —————
{
  "tracking_number": "1Z999AA10123456784",
  "carrier": "UPS",
  "status": "exception",
  "exception_detail": "Weather delay — Memphis hub",
  "original_eta": "2024-03-08",
  "revised_eta": "2024-03-13",
  "last_scan": "2024-03-09T18:45:00Z",
  "last_location": "Memphis, TN"
}

// — Warehouse Inventory —————
{
  "sku": "SKU-4892",
  "available": 120,
  "reserved": 50,
  "location": "Warehouse-East",
  "last_updated": "2024-03-10"
}

// — Quality Hold —————
{
  "sku": "SKU-4892",
  "hold_status": "none",
  "hold_reason": null,
  "hold_date": null,
  "expected_resolution": null
}

// — Contract Pricing —————
{
  "sku": "SKU-4892",
  "customer": "Acme Manufacturing",
  "unit_price": 495.00,
  "tier": "tier_2",
  "contract_id": "CONTRACT-ACME-2024",
  "contract_expiry": "2024-12-31"
}
```

What Just Happened?

The agent must navigate 5 different data sources to investigate one exception. The key insight: the investigation path is *dynamic*. A shipment delay starts with carrier tracking. A pricing discrepancy starts with contract verification. A quality hold

starts with the QA system. The ReAct agent reasons about which tool to call next based on intermediate findings.

Step-by-Step Build Guide

WHAT YOU WILL BUILD

A ReAct agent that investigates B2B order exceptions end-to-end. By the end of this guide, you will have a working multi-tool agent that diagnoses shipment delays, pricing discrepancies, partial deliveries, and quality holds — then drafts customer notifications.

Time estimate: 90 minutes – 2 hours | **Steps:** 8 numbered steps with code, run commands, expected output, and checkpoints

Step 1: Create the Project and Install Dependencies

What & Why: We create an isolated Python virtual environment so our dependencies do not conflict with other projects. We install the Anthropic SDK, which provides the `client.messages.create()` method for calling Claude with tools.

LINUX / MACOS · WINDOWS

LINUX / MACOS

```
mkdir exception-resolver && cd exception-resolver
python3 -m venv venv && source venv/bin/activate
pip install "anthropic≥0.30.0"
export ANTHROPIC_API_KEY=your-key-here
```

WINDOWS

```
# Run each line on its own – works in cmd.exe and PowerShell 5.1+
mkdir exception-resolver
cd exception-resolver
python -m venv venv
# PowerShell: venv\Scripts\Activate.ps1
# cmd.exe:    venv\Scripts\activate.bat
venv\Scripts\Activate.ps1
pip install "anthropic≥0.30.0"
# PowerShell: $env:ANTHROPIC_API_KEY = "your-key-here"
# cmd.exe:    set ANTHROPIC_API_KEY=your-key-here
$env:ANTHROPIC_API_KEY = "your-key-here"
```

Node.js / TypeScript Setup (optional)

Run command (Python):

```
python -c "import anthropic; print('SDK version:', anthropic.__version__)"
```

Expected output (any 0.30+ version is fine):

```
SDK version: 0.39.0
```

✅ Checkpoint

If you see a version number (0.30+ is fine), Step 1 is complete. If you see `ModuleNotFoundError`, make sure your virtual environment is activated and re-run `pip install "anthropic>=0.30.0"`.

File Structure

Here is the complete directory tree you will create over the next steps:

```
exception-resolver/
mock_tools.py      # All 6 mock tools (ERP, WMS, carrier, pricing, QA, comms)
agent.py           # ReAct agent loop with system prompt
mock_tools.ts      # Node.js version of mock tools
agent.ts           # Node.js version of agent
e2e_test.py        # End-to-end verification script
data/
  exceptions.json   # Sample exception inputs
  requirements.txt  # anthropic
  .env.example      # ANTHROPIC_API_KEY=
```

Step 2: Create the Mock Data

What & Why: Real B2B integrations talk to ERP, WMS, and carrier APIs that require credentials and complex setup. Instead, we create an in-memory mock database with realistic order data, tracking events, inventory levels, contract pricing, and quality hold records. This lets you build and test the full agent without any external dependencies.

Create/Edit: Create a new file called `data/exceptions.json`.

Run command (verify JSON is valid):

```
python -c "import json; data = json.load(open('data/exceptions.json')); print(f'{len(data)} exceptions loaded')"
```

Expected output:

```
3 exceptions loaded
```

✅ Checkpoint

If you see “3 exceptions loaded,” your data file is valid. If you see a `JSONDecodeError`, check for trailing commas or missing brackets in the JSON file.

Step 3: Build the 6 Mock Tools

What & Why: Each tool simulates a different backend system: ERP (order details), WMS (inventory), carrier tracking, contract pricing, quality hold status, and customer notification drafting. Each tool returns either a data record or an error dictionary. This is the same pattern you would use in production, except you would replace the mock dictionaries with real API calls.

Create/Edit: Create a new file called `mock_tools.py`. This is the complete file — paste the entire contents.

GOTCHA: The `PRICING_DB` uses a tuple `(sku, customer)` as the dictionary key. This is the lookup pattern for customer-specific contract pricing — the same SKU can have different prices for different customers.

Note: See the complete `mock_tools.py` code block below in the [Mock Tool Implementations](#) section — scroll down to copy it, then return here to verify the import.

Run command (verify all 6 tools import correctly):

```
python -c "from mock_tools import get_order_details, query_warehouse_inventory, track_shipment, get_contract_pricing, check_quality_hold_status, draft_customer_notification; print('All 6 tools imported successfully')"
```

Expected output:

All 6 tools imported successfully

✓ Checkpoint

If you see “All 6 tools imported successfully,” Step 3 is done. If you see an `ImportError`, check that `mock_tools.py` is in the same directory and has no syntax errors.

Step 4: Verify Each Tool Returns Correct Data

What & Why: Before building the agent, we verify each tool works independently. This isolates tool bugs from agent logic bugs. If the agent behaves unexpectedly later, you can rule out tool issues because you already verified them here.

Run command:

```
python -c "  
from mock_tools import *  
import json  
# Test each tool  
print('Order:', json.dumps(get_order_details('PO-2024-11234'))[:80], '...')  
print('Track:', json.dumps(track_shipment('1Z999AA10123456784', 'UPS'))[:80], '...')  
print('Inventory:', json.dumps(query_warehouse_inventory('SKU-4892'))[:80], '...')  
print('Pricing:', json.dumps(get_contract_pricing('Acme Manufacturing', 'SKU-4892'))[:80], '...')  
print('QA Hold:', json.dumps(check_quality_hold_status('SKU-4892'))[:80], '...')  
print('Not Found:', json.dumps(get_order_details('PO-FAKE-999'))[:80])  
"
```

Expected output (truncated):

```
Order: {"po_number": "PO-2024-11234", "customer": "Acme Manufacturing", "order_d ...  
Track: {"tracking_number": "1Z999AA10123456784", "carrier": "UPS", "status": "exc ...  
Inventory: {"sku": "SKU-4892", "available": 120, "reserved": 50, "location": "Wareh ...  
Pricing: {"sku": "SKU-4892", "customer": "Acme Manufacturing", "unit_price": 495.0 ...  
QA Hold: {"sku": "SKU-4892", "hold_status": "none", "hold_reason": null, "hold_dat ...  
Not Found: {"error": "NOT_FOUND", "message": "No order for 'PO-FAKE-999'."}
```

✓ Checkpoint

Each tool returns data for known inputs and an error dictionary for unknown inputs. The “Not Found” line confirms error handling works. If any tool throws an exception instead of returning a dictionary, check the function signature in `mock_tools.py`.

Step 5: Build the ReAct Agent Loop

What & Why: This is the core of the capstone. The agent loop sends a message to Claude, checks if Claude wants to call a tool (`stop_reason == "tool_use"`), executes the tool, returns the result, and repeats until Claude produces a final text response. The system prompt tells Claude to use the ReAct pattern: write a Thought before each tool call explaining its reasoning.

Create/Edit: Create a new file called `agent.py`. See the complete code block below in the [Complete Solution](#) section — scroll down to copy it, then return here to verify the import.

GOTCHA: The `while True` loop relies on `stop_reason == "tool_use"` to continue and `"end_turn"` to break. If the system prompt does not guide Claude to eventually stop calling tools and produce a final answer, the loop could run indefinitely. In production, add an iteration cap (e.g., `max_iterations = 15`).

✓ Checkpoint

You should now have two files: `mock_tools.py` (from Step 3) and `agent.py`. Both must be in the same directory. Run `python -c "from agent import investigate_exception; print('Agent module OK')"` to confirm the import works.

Step 6: Run the Agent on a Shipment Delay

What & Why: This is the moment of truth — we run the agent on a real exception input and see it reason through the investigation dynamically. The agent should pull order details, check carrier tracking, find the weather delay, verify no quality hold exists, and draft a customer notification.

Step dependency: This step uses `mock_tools.py` from Step 3 and `agent.py` from Step 5. Both files must exist in the same directory.

Run command:

```
python agent.py
```

When prompted, type `demo` to run the built-in sample exception (shipment delay for PO-2024-11234).

Step 7: Test Additional Exception Types

What & Why: A ReAct agent must handle different investigation paths for different exception types. We test a pricing discrepancy (which should start with contract verification) and a quality hold (which should start with the QA system). This verifies the agent's branching logic, not just one happy path.

Run `python agent.py` and paste each exception as JSON input (one at a time):

PRICING DISCREPANCY · **QUALITY HOLD**

PRICING DISCREPANCY

```
{"exception_id":"EXC-2024-0501","po_number":"P0-2024-11234","exception_type":"pricing_discrepancy","description":"Invoice shows $520/unit but contract says $495/unit for SKU-4892."}
```

QUALITY HOLD

```
{"exception_id":"EXC-2024-0523","po_number":"P0-2024-11890","exception_type":"quality_hold","description":"SKU-7721 on quality hold, cannot ship order P0-2024-11890."}
```

For pricing, the agent should: (1) pull order details, (2) check contract pricing, (3) identify the \$25/unit mismatch, (4) draft a pricing correction notification. For quality hold, the agent should: (1) check quality hold status for SKU-7721, (2) read order details, (3) report the expected resolution date, (4) draft a customer notification.

✓ Checkpoint

The agent should produce different tool call sequences for different exception types. A shipment delay triggers `track_shipment`; a pricing discrepancy triggers `get_contract_pricing`. If the agent always calls tools in the same order regardless of exception type, review your system prompt — the investigation strategies may need to be more explicit.

Step 8: Verify Error Recovery and Iteration Cap

What & Why: In production, carrier APIs go down, ERP systems timeout, and data can be missing. The agent must handle tool failures gracefully — note the failure, continue investigating with other tools, and flag the gap in its resolution. The `agent.py` you copied in Step 5 already includes an iteration cap (`MAX_ITERATIONS = 15`) to prevent infinite loops — in this step you confirm it works.

Inspect: Open `agent.py` and locate the `investigate_exception` function. Confirm you see the iteration counter and the bounded `while iteration < MAX_ITERATIONS` loop shown below.

ITERATION CAP (ALREADY IN AGENT.PY)

ITERATION CAP (ALREADY IN AGENT.PY)

```
# Inside investigate_exception (already present in agent.py):
MAX_ITERATIONS = 15
iteration = 0
WHILE iteration < MAX_ITERATIONS:
    iteration += 1
    response = client.messages.create(
        model=MODEL, max_tokens=2000,
        system=SYSTEM_PROMPT, tools=TOOLS,
        messages=history,
    )
    IF response.stop_reason == "tool_use":
        history.append({"role": "assistant", "content": response.content})
        history.append({"role": "user", "content": process_tool_calls(response)})
        CONTINUE
    history.append({"role": "assistant", "content": response.content})
    RETURN "\n".join(b.text FOR b IN response.content IF hasattr(b, "text"))

# If we exit the loop, the agent hit the iteration cap
RETURN "[WARN] Agent reached maximum iterations without resolving. Manual review required."
```

✓ Checkpoint

Try testing with a non-existent PO number (e.g., `"po_number": "PO-FAKE-999"`). The agent should get a `NOT_FOUND` error from the tool, reason about the failure, and stop gracefully instead of looping forever.

TROUBLESHOOTING

- If `agent.py` cannot import `mock_tools`, ensure both files are in the same directory and your terminal is `cd`'d into that directory.
- If the agent returns empty output, check that your `ANTHROPIC_API_KEY` is set: `echo $ANTHROPIC_API_KEY` (Unix) or `echo %ANTHROPIC_API_KEY%` (Windows).
- If the agent calls the same tool repeatedly, review the system prompt investigation strategies — ensure each exception type has a clear resolution path.

Mock Tool Implementations

PYTHON

PYTHON

```
ORDER_DB = {
    "PO-2024-11234": {
        "po_number": "PO-2024-11234", "customer": "Acme Manufacturing",
        "order_date": "2024-02-15", "status": "shipped", "total_value": 24750.00,
        "line_items": [
            {"line": 1, "sku": "SKU-4892", "description": "Industrial Valve Assembly - 4 inch",
             "qty_ordered": 50, "qty_shipped": 50, "qty_delivered": 0,
             "unit_price": 495.00, "tracking_number": "1Z999AA10123456784"},
        ],
    },
    "PO-2024-11890": {
        "po_number": "PO-2024-11890", "customer": "Summit Industrial Supply",
        "order_date": "2024-03-01", "status": "quality_hold", "total_value": 25000.00,
        "line_items": [

# ... (full source in the desktop HTML) ...

body = (f"Dear {customer},\n\n{summary}\n\n"
        f"Resolution: {resolution}{eta_line}\n\n"
        f"We apologize for any inconvenience. Please contact your account "
        f"manager IF you have questions.\n\nBest regards,\nOperations Team")
RETURN {"notification_id": f"NOT-{exception_type[:3].upper()}-001",
        "draft_text": body, "suggested_subject_line": subject}
```

Complete Solution

The system prompt is the key differentiator: it tells Claude to reason about the exception type before deciding which tools to call. Unlike Capstone 1 (always call one tool), the agent adapts its investigation path dynamically.

PYTHON

PYTHON

```
IMPORT json
IMPORT anthropic
FROM mock_tools import (
    get_order_details, query_warehouse_inventory, track_shipment,
    get_contract_pricing, check_quality_hold_status,
    draft_customer_notification,
)

client = anthropic.Anthropic()
MODEL = "claude-sonnet-4-6"

TOOL_HANDLERS = {
    "get_order_details": lambda a: get_order_details(a["po_number"]),
    "query_warehouse_inventory": lambda a: query_warehouse_inventory(

# ... (full source in the desktop HTML) ...

    print(f"\nAgent:\n{investigate_exception(cmd)}")
CATCH:
    print(f"\n[Error] {e}")

IF __name__ == "__main__":
    main()
```

What Just Happened?

You built a ReAct agent with 6 tools that dynamically investigates order exceptions. The system prompt provides investigation strategies per exception type, but the agent adapts in real time: if the carrier tracking reveals a quality hold, the

agent pivots to the QA system. After diagnosis, it always drafts a customer notification with the root cause and proposed resolution.

Expected Output — Shipment Delay Investigation

```
**Thought:** Exception EXC-2024-0456 is a shipment delay for
P0-2024-11234. Let me pull the order details first.

[Calls get_order_details → 50x SKU-4892 shipped via UPS]

**Thought:** Order was shipped. Let me check carrier tracking.

[Calls track_shipment → EXCEPTION: Weather delay, Memphis hub]

**Thought:** Weather delay at Memphis. Original ETA was Mar 8,
revised to Mar 13. Let me verify no quality hold exists.

[Calls check_quality_hold_status → No hold on SKU-4892]

**Thought:** Root cause confirmed: carrier weather delay.
Drafting customer notification with revised ETA.

[Calls draft_customer_notification]

**Resolution Summary:**
- **Root Cause:** UPS weather delay at Memphis hub
- **Impact:** 5-day delay (Mar 8 → Mar 13)
- **Action:** Customer notified with revised ETA
- **SLA Impact:** Force majeure – no credit warranted
- **Draft notification ready for review**
```

Verify Everything Works

Run this end-to-end verification script to confirm all components work together. Create a file called `e2e_test.py`:

E2E_TEST.PY

E2E_TEST.PY

```
# e2e_test.py – Verify all tools and agent components
FROM mock_tools import (get_order_details, query_warehouse_inventory,
    track_shipment, get_contract_pricing, check_quality_hold_status,
    draft_customer_notification)

# 1. ERP order lookup works
order = get_order_details("P0-2024-11234")
assert "error" not in order, "Order lookup failed"
assert order["customer"] == "Acme Manufacturing"
print("ERP:      OK – order found for Acme Manufacturing")

# 2. Carrier tracking works
tracking = track_shipment("1Z999AA10123456784", "UPS")
assert tracking["status"] == "exception"

# ... (full source in the desktop HTML) ...

# 7. Error handling works
not_found = get_order_details("P0-FAKE-999")
assert "error" in not_found
print(f"Errors:    OK – graceful error for unknown P0")

print("\nAll checks passed. Run 'python agent.py' and type 'demo' to test the full agent.")
```

Run command:

```
python e2e_test.py
```

Expected output:

```
ERP:      OK - order found for Acme Manufacturing
Carrier:  OK - status: exception (Weather delay - Memphis hub)
WMS:      OK - 120 units available at Warehouse-East
Pricing:  OK - $495.0/unit (tier: tier_2)
QA Hold:  OK - hold_status: none
Comms:    OK - draft created (NOT-shi-001)
Errors:   OK - graceful error for unknown PO

All checks passed. Run 'python agent.py' and type 'demo' to test the full agent.
```

 Congratulations

All 7 checks passed! Your Order Exception Resolution Agent is fully functional. You have a ReAct agent that dynamically investigates shipment delays, pricing discrepancies, partial deliveries, and quality holds across 6 different mock backend systems.

Testing Guide

TYPE	SCENARIO	EXPECTED BEHAVIOR
HAPPY	Shipment delay (weather)	Traces shipment, finds carrier exception, drafts notification with revised ETA
HAPPY	Pricing discrepancy on invoice	Compares order price vs contract, identifies mismatch, recommends credit
HAPPY	Partial delivery reported	Checks line items, finds backordered items, checks inventory for reshipment
HAPPY	Quality hold on SKU-7721	Discovers active hold, reports expected resolution date, notifies customer
HAPPY	Confirmed but not shipped order	Checks inventory availability, provides expected ship date
EDGE	Carrier API returns CARRIER_UNAVAILABLE	Notes failure, continues with ERP and warehouse data, flags for manual follow-up
EDGE	Exception has both pricing AND delay issues	Investigates both branches and addresses each in the resolution
EDGE	Customer has no contract on file	Falls back to list pricing comparison
ADVERSARIAL	Exception description blames company aggressively	Stays neutral, investigates facts, drafts professional response
ADVERSARIAL	Exception for non-existent PO	Reports NOT_FOUND, does not fabricate order details

Troubleshooting

Common issues and how to fix them:

```
ModuleNotFoundError: No module named 'anthropic'
```

You have not installed the Anthropic SDK. Make sure your virtual environment is activated (`source venv/bin/activate` on Unix, `venv\Scripts\activate` on Windows), then run `pip install "anthropic>=0.30.0"`.

AuthenticationError: Invalid API key

Your `ANTHROPIC_API_KEY` environment variable is not set or contains an invalid key. Verify with `echo $ANTHROPIC_API_KEY` (Linux/Mac) or `echo %ANTHROPIC_API_KEY%` (Windows). The key should start with `sk-ant-`. Re-export it: `export ANTHROPIC_API_KEY=sk-ant-...` (Unix) or `set ANTHROPIC_API_KEY=sk-ant-...` (Windows).

ImportError: cannot import name 'get_order_details' from 'mock_tools'

Both `mock_tools.py` and `agent.py` must be in the same directory. Check that you saved the file as `mock_tools.py` (underscores, not hyphens) and that it contains all 6 function definitions. Also verify your terminal is `cd` 'd into the project directory.

Agent loops without resolving (hits 15 iterations)

If the agent keeps calling tools without reaching a conclusion: (1) check that your `ORDER_DB` in `mock_tools.py` contains entries for the PO number you are testing, (2) verify the system prompt includes investigation strategies for the exception type, and (3) confirm the `draft_customer_notification` tool is in the `TOOLS` list so the agent can finalize its investigation. Lower the iteration cap to 10 during debugging to fail faster.

JSONDecodeError when pasting exception input

The agent expects valid JSON input. Make sure your exception JSON has no trailing commas, uses double quotes (not single quotes), and is pasted as a single line. Use the `demo` command first to verify the agent works before trying custom inputs.

Agent calls the same tool repeatedly

This usually means the system prompt is not guiding the agent to a conclusion. Check that the investigation strategy section includes explicit resolution paths (e.g., “After diagnosing root cause, ALWAYS draft a customer notification”). Also verify that tool results are being returned in the message history correctly — if tool results are missing, Claude will re-request the same tool.

Windows: 'python3' is not recognized

On Windows, use `python` instead of `python3`. If that also fails, verify Python is installed and added to your PATH: `python --version` should show 3.10 or higher. You can download Python from python.org.

Compliance Notes

⚠️ PCI-DSS & DATA SENSITIVITY

Exception investigations may expose sensitive commercial data: contract pricing, customer credit terms, inventory levels, and supplier quality issues. Production considerations:

- **Customer data isolation:** Exception investigations for Customer A must never expose Customer B’s contract pricing or order data. Enforce per-customer access controls at the tool level.
- **Quality hold confidentiality:** Quality hold information (supplier issues, defect details) is competitively sensitive. Customer notifications should state “product availability issue” rather than exposing supplier names or defect specifics.
- **Draft notification review:** All customer-facing drafts should be reviewed by a human before sending. The agent *drafts*; a human *approves and sends*.
- **Audit trail:** Every tool call in the investigation trace is an access event. Log with timestamp, user identity, and data accessed for compliance and SLA dispute resolution.

💰 COST OF REACT INVESTIGATIONS

Each exception investigation involves 4–6 tool calls with Thought traces. Typical consumption: 4,000–6,000 tokens per investigation. At Sonnet pricing, budget ~\$0.03–\$0.05 per exception. For 25 exceptions/day, that’s ~\$0.75–\$1.25/day. Compared to 20 minutes of analyst time per exception (\$8–\$12 in labor), the ROI is immediate.